



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

JAVA MULTITHREADING

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH



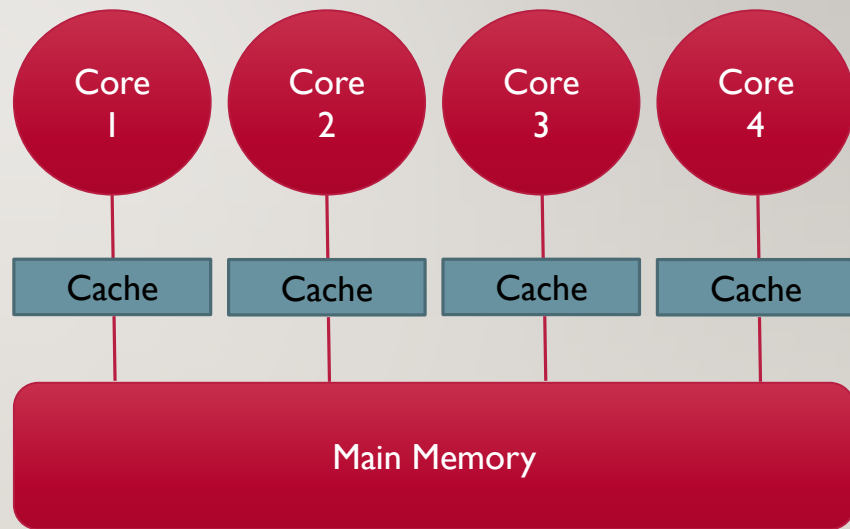
AGENDA

- Multithreading fundamentals
 - Why threads?
 - Thread lifecycle and states
- Two methods of thread creation in Java
 - Java's API for multithreading
 - Thread class
 - Runnable interface
- *Code resources*
 - See *Canvas*

BACKGROUND

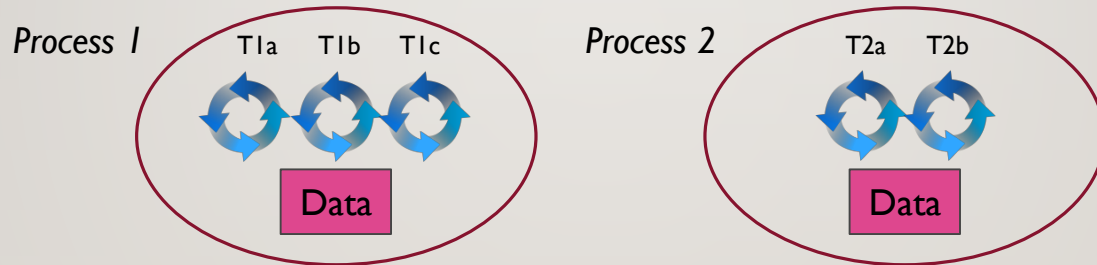
CENTRAL PROCESSING UNIT (CPU)

- **Central Processing Unit (CPU):** The CPU consists of multiple components to perform operations, we will focus on the cores and shared cache.
- **Core:** A core acts as a computation unit and provides a context for a *thread* to operate. Each core in a CPU may run **one** thread at a time. A core will usually have its own cache as well.
- **Process:** This is what starts when you run your Java program. When a process is started, resources are allocated by the OS for it to use. This starts with one executing thread.
- **Thread:** One or more threads exist within a process and can be executed to complete a task.
- **Task:** A set of program instructions that are defined by the written code.



MULTITHREADING

- A thread is a unit of concurrency within a process
- Threads can share data held within their process



- A thread scheduler determines when a particular thread executes
 - Scheduling is generally non-deterministic

WHY THREADS?

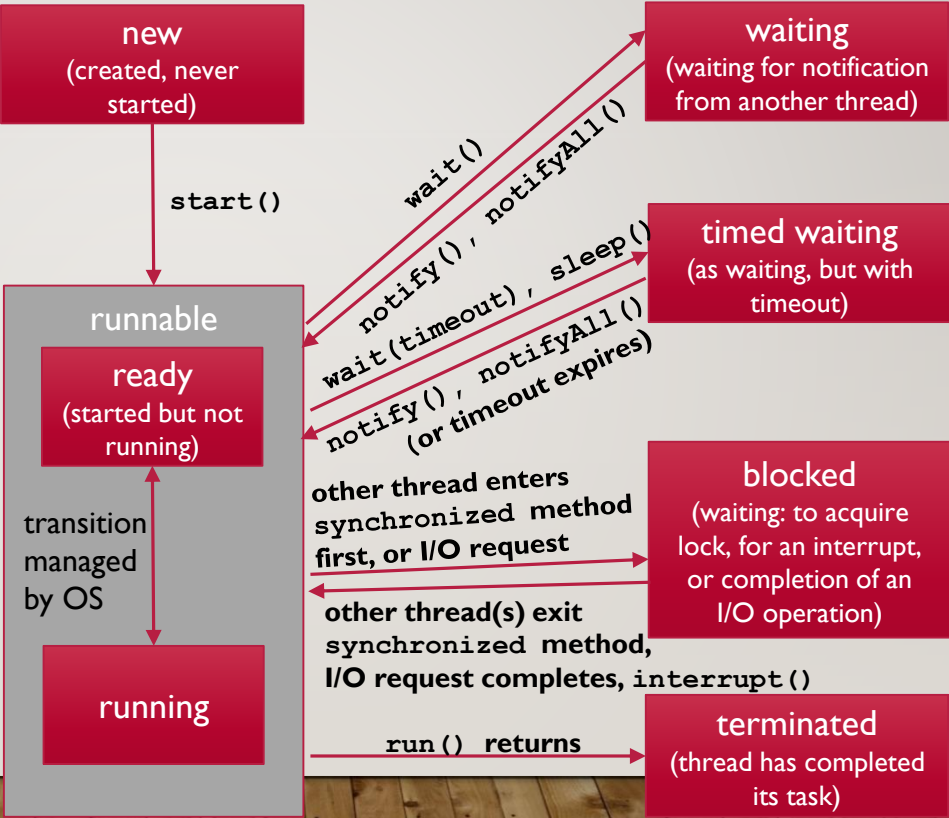
- **Efficiency:** Having just a single (OS level) thread means sticking to a single CPU/core. Other CPUs/cores remain idle.
 - Threads can use the other CPUs/cores: more efficient use of computing resource.
- **Responsiveness:** A single-threaded GUI application cannot respond to events while it is carrying out computing-intensive tasks in the background.
 - Multiple threads can dedicate one thread to the GUI and one or more to the backend (e.g., in the Model-View-Controller structure)
- **Task separation:** A single thread can only tend to do a single task at a time.
 - Multiple threads can interleave tasks

WHERE DO PEOPLE USE THREADS?

1. Server applications having to support multiple clients / sessions
2. Client applications operating multiple sessions or connections (e.g., web browsers)
3. Graphical user interfaces that are the frontend for computing- and/or I/O-intensive tasks
4. Image/video processing (operations on many pixels/blocks can be carried out in parallel on multiple CPUs)
5. Graphics / games: same reasons as 2 and 3 above
6. AI / machine learning
7. Search

...basically everywhere!

JAVA THREADS: LIFECYCLE



THREAD STATES

New: All threads begin in the New state. It remains this way until the program starts the thread, placing it in the Runnable state.

Waiting: A thread may place itself in a Waiting state when waiting for another thread to perform a task. It will remain this way until it is notified by another thread.

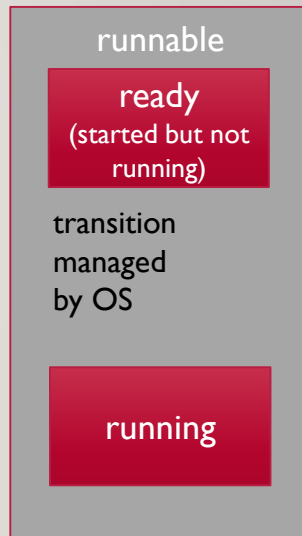
Timed Waiting: Specify a timeout interval where the thread will stop waiting and switch back to the Runnable state. Consider a program backing up a hard drive for waiting uses.

Blocked: A thread in the Runnable state will switch to Blocked when it attempts to perform a task that it can't perform immediately. It must wait until the other task blocking it completes before it returns to Runnable.

Terminated: A thread moves to the Terminated state when it successfully completes its task or it otherwise terminates, say due to an error.

OS VIEW OF **R**UNNABLE STATE

- At the OS level, the Runnable state actually encompasses two states, **ready** and **running**. These are hidden from the JVM which only views the Runnable state.
- When a thread enters the Runnable state, at the OS level it starts in the ready state. It will move to the running state once it has been assigned a time slice on a processor by the OS. This is known as **dispatching the thread**.
- The process of determining how the thread will be dispatched is known as **thread scheduling**.



THREAD CLASS

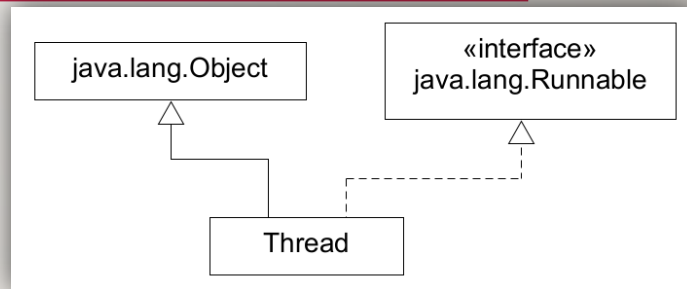
How can we create a thread to execute our code on?

Method 1:

We can use the native **Thread** class.

In order to utilize this class, there are a **few steps** we must go through:

- Define a new class that extends the `Thread` class
- Override the `run()` method of the `Thread` superclass
- Within the `run()` method we place our code to be executed by the thread. i.e. our 'Task'
- Instantiate a new object of our custom class derived from the `Thread` class
- Call the `start()` method on the instantiated object



METHOD I: USING THREAD CLASS

EXAMPLE

- Our custom class `MyThread` is declared and extends the Java `Thread` class.
- We override the `run` method, and declare the logic for the intended task that the thread must perform.
- After defining our `MyThread` class we are able to instantiate this thread and pass in a name through the constructor at instantiation.
- The instantiated thread can then have the `start()` method called on it. Invoking this method will cause the thread to enter the *Runnable* state and execute the code defined inside the overridden `run()` method
- The **`Thread`** class has a variety of useful methods which can be used to retrieve the properties of the thread. The **`getState()`** method will indicate which of the 6 states (*new*, *runnable*, *waiting*, *timed waiting*, *blocked*, *terminated*) the thread is in.

```
3 public class MyThread extends Thread {
4
5     public MyThread(String name) {
6         super(name);
7     }
8
9     @Override
10    public void run() {
11        //Our Task
12    }
13
14 }
```

```
3 public class TestMyThread {
4
5     public static void main(String[] args) {
6
7         MyThread thread1 = new MyThread("computation");
8         thread1.start();
9     }
10
11
12 }
```

THREAD CLASS – A GLIMPSE

- Class Thread defines several methods that include:
 - **isAlive()** returns true if the Thread's runnable object hasn't completed its processing, false otherwise
 - **interrupt()** requests that a thread terminates gracefully
 - **interrupted()** a class (static) method that returns true if the currently executing thread has been interrupted, false otherwise
 - **join()** synchronises the calling thread with the death of the Thread object upon which join() is called
 - **start()** causes a Thread to execute its Runnable object
 - **stop()** forcefully terminates a Thread; any monitor locks held by the Thread are freed
- **stop()** should never be used (it's actually deprecated)
- **wait()** and **notify()** methods are inherited from Object class and also enable thread synchronization alike **join()** method.

SIMPLETHREAD CLASS

- The snippet above shows a new custom class **SimpleThread** which extends `Thread`.
- We have implemented a very simple task, to print the name of the thread 10 times, and upon completion print "DONE!" followed by the task's name.
- We can now instantiate two threads, both of the `SimpleThread` class, and call their `start()` method.
- **How will the threads' tasks be executed?**

```
3 public class SimpleThread extends Thread {
4
5     public SimpleThread(String name) {
6         super(name);
7     }
8
9     public void run() {
10         for (int i=0; i<10; i++) {
11             System.out.println(i + " " + getName());
12         }
13         System.out.println("DONE! " + getName());
14     }
15 }
```

```
3 public class TestSimpleThread {
4
5     public static void main(String[] args) {
6
7         SimpleThread thread1 = new SimpleThread("first");
8         SimpleThread thread2 = new SimpleThread("second");
9
10        thread1.start();
11        thread2.start();
12
13    }
14
15 }
```

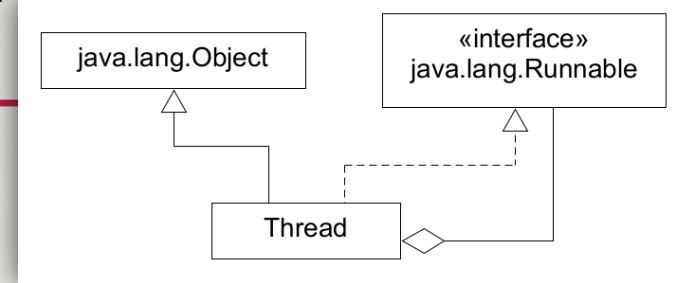
SIMPLETHREAD CLASS

- The output from each thread is intermingled with the output of the other thread.
- The order is **not** predictable. If we re-run the simulation we will see different ordering results.
- **For this example**, there are **no inconsistencies with the data**. However, if the threads attempt to use any shared data we must take care to use proper **synchronization**.

0	4	7	DONE!
second	second	second	second
1	2 first	5 first	8 first
second	5	8	9 first
0 first	second	second	DONE! first
2	3 first	6 first	
second	6	9	
3	second	second	
second	4 first	7 first	
1 first			

METHOD 2: *RUNNABLE* INTERFACE

- There is another option for creating a thread to execute a task on.
- This involves using the native **Runnable** interface.
- Implementing the **Runnable** interface is similar to extending the **Thread** class, but there are some key differences in how we instantiate the class implementing **Runnable**.
 - Define a new class that implements the **Runnable** interface
 - Implement the **run ()** method of the **Runnable** interface
 - Within the **run ()** method we place our code to be executed by the thread. i.e. our 'Task'
 - **Instantiate a **Thread** with our custom runnable in the constructor, along with the thread name.**
 - Call the **start ()** method on the instantiated **Thread**



RUNNABLE INTERFACE

- Our custom class **SimpleRunnable** is declared and implements the Java **Runnable** class.
- We implement the **run** method (as it is declared in the interface), and code the logic for the intended task that the thread must perform.
- After defining our **SimpleRunnable** class we are able to instantiate a **Thread** object by passing a **SimpleRunnable** instance, along with the desired thread name, into the constructor.
- The instantiated thread can then have the **start()** method called on it. Invoking this method will cause the thread to enter the **Runnable** state and execute the code defined inside the implemented **run()** method

```
3 public class SimpleRunnable implements Runnable {
4
5     public void run() {
6         //Do Task
7     }
8 }
```

```
3 public class TestSimpleRunnable {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7
8         Thread thread1 = new Thread(new SimpleRunnable(), "first");
9         thread1.start();
10
11     }
12
13 }
```

RUNNABLE INTERFACE

- The task defined in the `run()` method of our `SimpleRunnable` class will now be run independently of the main thread that created it.
- It is important to note that a `Thread` instance is still required. The runnable itself simply defines the `run()` method and must be passed into a `Thread` object to be treated as a thread.
- Without the thread, we may only use the `SimpleRunnable` as a way of invoking the `run()` method in sequential format.
- As the `SimpleRunnable` isn't instantiated as a thread, it doesn't have a reference to the `Thread` that is executing its task.
- This makes it more difficult to access `Thread` properties than it was for `SimpleThread`. For the `SimpleRunnable` to get a reference to the thread, it must call the static method `currentMethod()` from the `Thread` class. We can then access the thread properties with **getters** and **setters** as previously.

```
3 public class SimpleRunnable implements Runnable{
4
5     public void run() {
6
7         String name = Thread.currentThread().getName();
8
9         for (int i=0; i<10; i++) {
10             System.out.println(i + " " + name);
11         }
12         System.out.println("DONE! " + name);
13     }
14 }
```

EXTENDING THREAD VS IMPLEMENTING RUNNABLE

We have defined two different ways to successfully execute logic on its own thread, allowing us to run multiple threads and execute multiple tasks, at the same time.

Which method is better to use?

Implementing Runnable is the preferred method in most cases.

- **Java only allows a class to extend one superclass.** If we extend thread it prevents us from extending a more appropriate superclass. (OOP tells us Implement over extend where possible!)
- **The Runnable instance can be reused over multiple threads**, reducing the overall number of objects created. This lowers memory consumption

EXAMPLE: A TASK TO COMPUTE PRIME

```

public class PrimeFactorsTask implements Runnable {
    private long _n;

    public PrimeFactorsTask( long n ) {
        _n = n;
    }

    public void run() {
        System.out.print( "The prime factorization of " + _n + " is: ");

        for (long i = 2; i*i <= n; i++) {
            while (n % i == 0) {
                System.out.print(i + " ");
                n = n / i;
            }
        }

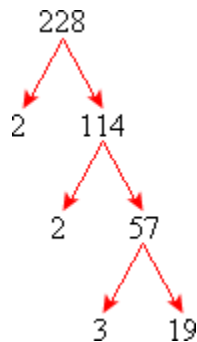
        if (n > 1) System.out.println(n);
        else      System.out.println( );
    }
}

```

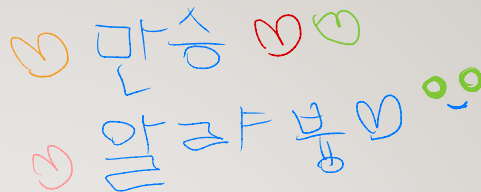
```

public static void main(String[] args) {
    Runnable task = new PrimeFactorsTask( 9201111169755555649L );
    Thread worker = new Thread( task );
    worker.start();
}

```



REVIEW



- **Learnt something about:**
 - **Fundamental multithreading**
 - Using threads creates the illusion of concurrent work by interleaving their execution
 - Multithreaded programming introduces new kinds of problems, e.g. ensuring that threads don't corrupt shared data, ensuring that deadlock situations are avoided
 - Java's support for multithreading lies in **Thread** and **Runnable**
 - A **Thread** object is a thread that competes for processor time
 - A **Thread** object holds a reference to some kind of **Runnable** object
 - When started, via **start()**, a **Thread** object calls its **Runnable** object's **run()** method
 - The **Thread** object "dies" when its **Runnable** object's **run()** method terminates

REFERENCES

- *Books:*
 - *Book:* P. Deitel and H. Deitel, Java How to Program
- *Other:*
 - <https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>